Reinforcement learning draws on animal psychology, game-playing programs, and optimal control theory in roughly equal measure. Thorndike's law of effect and behaviourist trial-and-error learning provided the notion of "reinforcement" as formalized by Rescorla-Wagner. Chess and checkers programs of Shannon and Samuel from the 1950s onward gave researchers concrete problems on which to test ideas about machine learning. The Bellman-Howard-Blackwell dynamic programming framework provided the recursive structure and language.

0.1 Animal Psychology

Controlled experiments on rats, cats, and dogs inspired the "gridworld" environments still used today, and shaped how the field conceptualizes "training" agents through "reward signals". Thorndike (1898) placed cats in puzzle boxes with latched doors and food visible outside. Across 15 different box configurations, the cats initially engaged in undirected behavior such as clawing at the walls, pushing against the bars, reaching through openings. The first cat to escape the simplest box required 160 seconds of random activity before accidentally pressing the latch. By the 24th trial, the same cat pressed the latch directly within 6 seconds. The learning curves showed gradual, continuous improvement rather than sudden insight. From these experiments the Law of Effect was formulated: responses (actions) followed by satisfaction (positive rewards) are "stamped in" and more likely to recur, while those followed by discomfort (negative rewards) are "stamped out."

Pavlov (1927) noted that dogs salivated not only at food itself but at the sight of an empty food bowl, the sound of footsteps, and other stimuli that preceded feeding (i.e. the state). To measure these "psychic secretions" precisely, he surgically implanted fistulas (tubes allowing external collection) to collect saliva. In the canonical experiment, a metronome sounded before food delivery. After 20–40 pairings, the metronome alone elicited salivation. The response followed not from the stimulus itself but from what it "predicted". In reinforcement learning terms, the conditioned stimulus is a state s, and the learned expectation of food is the value function $V(s)$.

Kamin (1969) demonstrated that learning requires more than mere co-occurrence in time. In Phase I of his blocking experiment, rats learned that a noise predicted a shock and developed a conditioned fear response to the noise alone. In Phase II, a compound stimulus of noise plus light was paired with the same shock. When the light

was subsequently presented alone, no fear response occurred. The light was "blocked" (ignored) because the noise already predicted the shock perfectly. There was no prediction error (or "surprise") to drive learning about the light. Once the noise was established as a predictor, the light added no new information and thus no new learning occurred.

Rescorla and Wagner (1972) formalized the blocking phenomenon as a prediction-error learning rule. Translating to RL notation:[1]

$$V(s) \leftarrow V(s) + \alpha\,\delta, \quad \text{where } \delta = r - V(s) \tag{1}$$

This is temporal difference learning with $\gamma = 0$, without discounting of future rewards. Each stimulus s begins with $V(s) = 0$. On each trial, the organism observes the stimuli present, receives outcome $r \in \{0, 1\}$, and updates values according to δ. The model is purely predictive; there are no actions, only learned expectations about reward.[2] The model's power lay in prediction, not just explanation. It correctly predicted overexpectation, whereby two separately conditioned stimuli combined and reinforced together each lose value because their summed prediction exceeds r.

0.2 Board Games

Chess and checkers are sequential decision problems. The board position is a state $s \in \mathcal{S}$, a legal move is an action $a \in \mathcal{A}(s)$, and the resulting position is a successor state $s' = T(s, a)$ determined by the rules of the game. The game outcome provides a terminal reward $r \in \{+1, 0, -1\}$ for win, draw, or loss. An evaluation function $f(P)$ that scores a position corresponds to a value function $V(s)$ estimating expected outcome. These games are deterministic (no chance moves in chess), fully observable (both players see the entire board), and zero-sum (one player's gain is the other's loss). The adversarial structure introduces a second player whose actions a' must be anticipated.

Shannon (1950) posed the fundamental question: can we program a general-purpose computer to play chess, and if so, what principles should guide the design? The challenge is computational. A chess game averages 40 moves per player with roughly 30 legal

[1] The original notation used V_i for associative strength of stimulus i, α_i for stimulus salience (noticeability), β_j for learning rate, λ_j for maximum conditioning (1 if reward present, 0 otherwise), and $V_{\text{tot}} = \sum_k V_k$ for total prediction. The correspondence is: $V_i \to V(s)$, $\alpha_i\beta_j \to \alpha$, $\lambda_j \to r$, $V_{\text{tot}} \to V(s)$. See Sutton and Barto (1990) for details.

[2] The Rescorla-Wagner update is mathematically identical to the Widrow-Hoff least mean squares rule.

moves available at each position. Shannon estimated that exhaustive search through all possible games would require examining approximately $30^{80} \approx 10^{120}$ positions, a number exceeding the atoms in the observable universe. This is the curse of dimensionality applied to games, where state space size grows exponentially with the number of sequential decisions. Shannon calculated that brute-force enumeration would require 10^{90} years at any foreseeable computing speed. The curse intensifies with game complexity. Chess has approximately 10^{47} legal positions, shogi 10^{71}, and Go 10^{171}.[3]

Shannon distinguished two approaches. Type A strategies search all continuations to a fixed depth H, building a complete game tree and evaluating every leaf ("rote-learning"). Type B strategies search selectively, examining only variations deemed important by some criterion ("generalization"). Either approach requires an evaluation function $f(P)$ to score positions where search terminates. Shannon proposed linear evaluation:

$$f(P) = \sum_i w_i \phi_i(P) \tag{2}$$

with features ϕ_i for material, mobility, pawn structure, and king safety. Weights w_i were hand-tuned. The minimax principle governs adversarial search. In a two-player zero-sum game, the value of a position satisfies

$$V(s) = \max_{a \in \mathcal{A}(s)} \min_{a' \in \mathcal{A}'(s')} V(T(s, a, a')) \tag{3}$$

where the maximizing player moves first and the minimizing opponent responds optimally. This is model-based planning, since the transition function T is known exactly from the game rules. The computational problem is how to use limited search resources effectively given the exponential tree.

Both Type A and Type B strategies truncate the game tree at depth H and substitute the evaluation function for exact continuation values. This is approximate dynamic programming. The true value $V^*(s)$ satisfies a recursive equation, but computing it exactly is infeasible, so the recursion is truncated and terminal values approximated. Deeper lookahead (H-step search) builds larger trees; rollout policies extend search by simulating play to the end using a fast base policy.[4] The evaluation function serves as a heuristic

[3] State space estimates from Igami (2020).

[4] Monte Carlo tree search, developed later, samples rollouts rather than enumerating all branches, enabling deeper effective lookahead in games with large branching factors.

substitute for exact computation. Shannon did not implement a chess program; the 1950 paper is theoretical, outlining the architecture that shaped fifty years of game engines.

Samuel (1959) built a checkers program for the IBM 704 that could improve through experience. The program played against itself, generating virtually unlimited training data at no cost. It parameterized the value function as a linear combination of hand-crafted features (piece advantage, mobility, king safety):

$$V(s; \mathbf{w}) = \sum_i w_i \phi_i(s) \tag{4}$$

After each move from s_t to s_{t+1}, weights were updated by temporal difference:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[V(s_{t+1}; \mathbf{w}) - V(s_t; \mathbf{w}) \right] \nabla_{\mathbf{w}} V(s_t; \mathbf{w}) \tag{5}$$

In a 1965 match, World Champion W.F. Hellman won all four games played by mail, but was played to a draw in one game. After learning from 173,989 book moves, the program agreed with the book-recommended move (or rated only 1 move higher) 64% of the time without lookahead. With lookahead and minimaxing, it followed book moves "a much higher fraction of the time."

The linear parameterization compresses the value function from 10^{20} table entries to dozens of weights, the essential response to the curse of dimensionality. Samuel's architecture — minimax search to depth H with the learned evaluation function scoring leaves — is an early instance of rollout, where tree search simulates forward, truncating at H and substituting $V(s)$ for exact continuation values.[5] The conceptual apparatus of modern game-playing AI — self-play, evaluation learning, tree search, and function approximation — was present in the 1950s.

0.3 Optimal Control

Bellman (1957) considered multi-stage decision processes in which a system occupies state $s \in \mathcal{S}$, the decision-maker chooses action $a \in \mathcal{A}$, the system transitions to $s' \sim P(\cdot|s, a)$, and a reward $r(s, a, s')$ accrues. The objective is to maximize cumulative reward over a finite or infinite horizon. The classical approach treats an N-stage process as a single

[5] Bertsekas (2021) interprets this as approximate dynamic programming, where offline training (learning V) combined with online planning (tree search) implements a Newton-like step for the Bellman equation.

N-dimensional optimization. Bellman calculated the consequence. A 10-stage process with 10 grid points per variable requires 10^{10} function evaluations; at one evaluation per second, 10^{10} evaluations require 2.77 million hours. He called this exponential growth the curse of dimensionality. His solution was the principle of optimality, namely that an optimal policy has the property that, whatever the initial state and initial decision, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision. This principle yields the Bellman equation:

$$V^*(s) = \max_{a \in \mathcal{A}} \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^*(s') \right] \tag{6}$$

The equation reduces an N-dimensional problem to a sequence of N one-dimensional problems. Value iteration computes V^* by iterating $V_{k+1}(s) = \max_a [r(s,a) + \gamma \sum_{s'} P(s'|s,a) V_k(s')]$. The monograph applied the method to resource allocation, inventory control, bottleneck scheduling, gold mining under uncertainty, and multi-stage games.[6]

Howard (1960) observed that value iteration converges slowly for problems of indefinite duration. His alternative, policy iteration, solves for the value function of a fixed policy and then improves the policy directly. Given policy π, policy evaluation computes the gain g (average reward per period) and relative values v_i by solving the linear system $g + v_i = q_i + \sum_j p_{ij} v_j$ for $i = 1, \ldots, N$, where q_i is the expected immediate reward in state i and p_{ij} is the transition probability under π. Policy improvement then selects, for each state i, the action k maximizing $q_i^k + \sum_j p_{ij}^k v_j$. Howard proved that each iteration strictly increases the gain unless the policy is already optimal, and the algorithm terminates in finitely many steps. For a problem with 50 states and 50 actions per state, exhaustive enumeration must consider $50^{50} \approx 10^{85}$ policies; policy iteration finds the optimum in a handful of iterations. Howard demonstrated the method on a toymaker's production problem, taxicab dispatch in three city zones, and automobile replacement timing.

Blackwell (1965) established the measure-theoretic foundations for discounted dynamic programming with general state and action spaces. He proved that the Bellman operator T defined by $Tu(s) = \sup_a [r(s,a) + \gamma \int u(s') P(ds'|s,a)]$ is a contraction with modulus γ: $\|Tu - Tv\|_\infty \le \gamma \|u - v\|_\infty$. Banach's fixed-point theorem then guarantees a unique bounded solution V^* to the Bellman equation, with $\|V_k - V^*\|_\infty \le \gamma^k \|V_0 - V^*\|_\infty$

[6] Chapter IX shows how dynamic programming derives classical variational conditions from the functional equation. The continuous-time analogue is the Hamilton-Jacobi-Bellman equation.

under value iteration. The central result concerns stationary policies, which use the same decision rule $f : \mathcal{S} \to \mathcal{A}$ at every period regardless of history. Blackwell proved that if the action space is finite, there exists an optimal stationary policy. For countable action spaces, ϵ-optimal stationary policies exist for every $\epsilon > 0$. These results justify the focus on memoryless policies, since optimal behavior depends only on the current state, not on the history of past states and actions. The Bellman equation, Howard's policy iteration, and Blackwell's existence theorems constitute the planning framework. Given complete knowledge of P and r, these methods compute optimal policies exactly. The challenge of learning without such knowledge is the central problem of reinforcement learning.

0.4 Monte Carlo Estimation

Monte Carlo estimation addresses this challenge by replacing exact computation with sampling. Given a policy π, the agent generates complete episodes $(s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_T)$ and computes the realized return from each visited state:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \tag{7}$$

First-visit Monte Carlo prediction averages G_t across episodes for each state s, counting only the first occurrence of s in each episode. By the law of large numbers, this average converges to $V^\pi(s)$ as the number of episodes grows (Sutton and Barto, 2018). The incremental form of the update is $V(s) \leftarrow V(s) + \alpha[G_t - V(s)]$, the same Rescorla-Wagner prediction-error structure from Section 1.1, with the full episode return G_t replacing the immediate reward r.

Monte Carlo methods are model-free: they require only sampled trajectories, not knowledge of P or r. But they carry two limitations. First, the agent must wait until an episode terminates to compute G_t, so the method cannot learn from incomplete sequences or continuing tasks. Second, G_t accumulates randomness from every transition along the trajectory, producing high-variance estimates that require many episodes to stabilize. The planning framework gives exact solutions when the model is known; Monte Carlo gives unbiased estimates when it is not. Neither method uses current value estimates of successor states to update the value of the current state. Temporal difference learning, introduced next, combines the model-free property of Monte Carlo with the incremental,

bootstrapping structure of dynamic programming.

References

Richard Bellman. *Dynamic Programming.* Princeton University Press, 1957.

Dimitri P. Bertsekas. Lessons from AlphaZero for optimal, model predictive, and adaptive control. *Athena Scientific Reports*, 2021.

David Blackwell. Discounted dynamic programming. *Annals of Mathematical Statistics*, 36(1):226–235, 1965.

Ronald A. Howard. *Dynamic Programming and Markov Processes.* The Technology Press of M.I.T. and John Wiley and Sons, New York, NY, 1960.

Mitsuru Igami. Artificial intelligence as structural estimation: Deep blue, bonanza, and alphago. *The Econometrics Journal*, 23(3):S1–S24, 2020.

Leon J. Kamin. Predictability, surprise, attention and conditioning. In Byron A. Campbell and Russell M. Church, editors, *Punishment and Aversive Behavior*, pages 279–296. Appleton-Century-Crofts, 1969.

Ivan P. Pavlov. *Conditioned Reflexes: An Investigation of the Physiological Activity of the Cerebral Cortex.* Oxford University Press, 1927.

Robert A. Rescorla and Allan R. Wagner. A theory of Pavlovian conditioning: Variations in the effectiveness of reinforcement and nonreinforcement. In Abraham H. Black and William F. Prokasy, editors, *Classical Conditioning II: Current Research and Theory*, pages 64–99. Appleton-Century-Crofts, 1972.

Arthur L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959.

Claude E. Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 41(314):256–275, 1950.

Richard S. Sutton and Andrew G. Barto. Time-derivative models of Pavlovian reinforcement. In Michael Gabriel and John Moore, editors, *Learning and Computational Neuroscience: Foundations of Adaptive Networks*, pages 497–537. MIT Press, 1990.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2nd edition, 2018.

Edward L. Thorndike. *Animal Intelligence: An Experimental Study of the Associative Processes in Animals*. Macmillan, 1898. Psychological Review Monograph Supplements, No. 8.